```
# Solução do primeiro problema da aulaX.
# JMR@ua.pt Dez 2018
#
# O problema foi resolvido por fases sucessivas e sugere-se que seja
analisado
# pela mesma ordem.
#
# 1) Primeiro foi criada a função principal, main(), com o ciclo, menu e
ifs
# para tratar a opção escolhida, inicialmente apenas com instruções
vazias.
# Apenas se implementou a opção 5, para terminar, e o aviso de opção
errada no
# else.  Esses são simples.
# O menu poderia ter sido escrito com vários prints, mas aqui
implementámos
# simplesmente como um "prompt" de várias linhas do input().  Porque não?
#
# 2) Desenvolvida a função isPhone.  Foi testada com algumas instruções
# assert para verificar um conjunto de resultados desejados.  Foi escrita
# por fases, primeiro para verificar apenas telefones formados por
dígitos,
# que corresponde ao método .isdigit já existente nas stings.
# Depois acrescentámos as restantes condições de validade.
# Usámos regras da lógica para reorganizar e simplificar a condição.
#
# 3) Implementar a opção 1 de ler/registar uma chamada.
# Implicou acrescentar código após o if op == "1".
# Como é uma operação algo complexa, inventámos inputCall para ler uma
# chamada para depois a registar na estrutura de dados chamadas.
#
# 4) Ao definir inputCall, era preciso ler a origem e o destino, ambos
# números de telefone que deveriam ser validados.  Para evitar
copy&paste,
# fazia sentido inventar inputPhone: uma função para ler e validar um
número
# de telefone.  Invocando essa função, inputCall fica muito simples.
# Como vamos precisar de fazer cálculos com a duração, faz sentido ler e
# converter imediatamente para int.
# inputCall devolve os três valores na forma de um tuplo que representa a
# chamada, e é sempre assim que as chamadas serão representadas em todo o
# programa.
#
```

```
# 5) Para ser geral, inputPhone tem de receber o prompt como parâmetro.
# Assim pode ser usada para pedir a "origem" ou o "destino".
# É implementada de uma forma típica para uma leitura com validação:
# Basta repetir a leitura e sair (break) apenas quando o valor lido
satisfaz a
# condição desejada.  A condição neste caso é `isPhone(tel)`,
simplesmente.
#
# 6) No main, a variável chamadas terá que registar as sucessivas
chamadas
# que forem inseridas pelo utilizador.
# Numa primeira abordagem, poderia usar-se uma lista à qual se fosse
# acrescentando cada chamada, mas mais à frente vamos ver que precisamos
de
# encontrar todas as chamadas de um certo cliente, e de saber que
clientes
# temos.
# Tudo isso fica facilitado e mais _eficiente_ se chamadas for um
dicionário
# que a cada cliente (chave) associe uma lista das chamadas desse
cliente.
# Ou seja, algo como: {"96245": [("96245", "234882", 1.23), ...], ... }.
#
# Esta estrutura de dados é _fundamental_ no programa.
# É preciso compreender bem como a usar corretamente.
# A forma de acrescentar uma nova chamada é o problema principal.
# Optámos por implementá-la numa função addCall.  Vai ser reutilizada!
# Como se trata de um dicionário, é preciso distinguir se a chamada
# é de um cliente "novo" ou de um que já tem registo.
# E lembrar que o valor a associar tem de ser uma lista,
# ainda que com apenas uma (ou nenhuma) chamada.
#
# 7) Implementar a opção 2 de leitura do ficheiro agora é simples.
# É feito na função loadCalls que recebe a referência para o dicionário
# chamadas e altera-o pelo mesmo processo que na opção 1, mas agora com
cada
# chamada extraída linha-a-linha do ficheiro.
#
# 8) A opção 3 para mostrar os clientes fica trivial devido à opção
tomada no
# ponto (6).  Os clientes são as chaves do dicionário.  Basta ordená-las
com
# sorted e mostrá-las.
#
```

```
# 9) A opção 4 para mostrar a fatura de um cliente é implementada na
função
# fatura.  Os parâmetros necessários são o cliente e o registo de
chamadas,
# claro.  Com o dicionário obtém-se a lista das chamadas desse cliente
# diretamente.  O problema agora é só percorrer essa lista para mostrar a
# informação necessária de cada chamada feita: destino, duração e custo.
# Imprimir na forma de tabela é apenas uma questão de usar as facilidades
de
# alinhamento do método .format.
# O problema de calcular o custo da chamada é separado num nova função.
#
# Para o programa ficar completo, falta corrigir e completar uns detalhes
que
# se deixam como exercícios, indicados por comentários #TPC.
#
# Bom estudo!

def isPhone(tel):
    """Indica se tel é um número de telefone válido."""
    r = (len(tel)>=3 and tel.isdigit()) or \
        (len(tel)>=4 and tel[0]=="+" and tel[1:].isdigit())
    return r

# Testes unitários à função isPhone
# Estes testes foram feitos para testar a função anterior, enquanto era
desenvolvida.
# Quando correta, nenhum assert falha (e poderiam ser retirados)
assert isPhone("24352") == True
assert isPhone("52") == False
assert isPhone("+252") == True
assert isPhone("+23") == False
assert isPhone("+23") == False
assert isPhone("") == False

def inputPhone(prompt):
    """Pede e garante que lê um número de telefone válido."""
    while True:
        tel = input(prompt)
        if isPhone(tel): break
        print("Telefone inválido")  # (Não pedia esta mensagem, mas seria
assim.)
    return tel
```

```
def inputCall():
    """Lê os dados (validados) de uma chamada. Devolve num tuplo."""
    origem = inputPhone("Tel origem? ")
    destino = inputPhone("Tel destino? ")
    # se fosse pedido, também poderíamos validar a duracao (garantir que
>=0, por exemplo)
    duracao = int(input("Duração? "))
    return (origem, destino, duracao)

def addCall(chamada, chamadas):
    """Acrescenta uma nova chamada à estrutura de dados."""
    cliente = chamada[0]
    if cliente not in chamadas:       # se é cliente "novo", ...
        chamadas[cliente] = []        # ... associa-lhe uma lista vazia
    chamadas[cliente].append(chamada) # em qq caso, acrescenta chamada à
lista

def loadCalls(fname, chamadas):
    """Lê um ficheiro de chamadas (fname) e junta-as ao registo
(dicionário chamadas)."""
    f = open(fname, "r")
    for line in f:
        #print(repr(line))
        parts = line.strip().split()
        chamada = (parts[0], parts[1], int(parts[2]))
        addCall(chamada, chamadas)
    f.close()


import math

def custoChamada(chamada):
    """Avalia o custo de uma chamada, em função do destino e duração."""
    origem, destino, duracao = chamada
    if destino[0] == "2":
        cmin = 0.02
    elif destino[0] == "+":
        cmin = 0.80
    elif destino[:2] == origem[:2]:
        cmin = 0.04
    else:
        cmin = 0.10
    c = duracao*cmin/60.0
    return math.ceil(c*100.0)/100.0   # arredonda ao cêntimo acima (não
era pedido)
```

```
def fatura(cliente, chamadas):
    """Imprime a fatura de um cliente."""
    print("Fatura do cliente: {}".format(cliente))
    # Escrever cabeçalho da tabela, com títulos devidamente alinhados
    print("{:<20s} {:>8s} {:>10s}".format("Destino", "Duração", "Custo"))
    lst = chamadas[cliente]
    for ch in lst:
        custo = custoChamada(ch)  # falta calcular custo
        print("{:<20s} {:>8d} {:>10.2f}".format(ch[1], ch[2], custo))
    # TPC: Falta calcular e imprimir o total da fatura.


# MENU é uma string com várias linhas.
# (Foi definida como global porque torna o main mais legível e,
# sendo uma constante, não traz quaisquer problemas.)
MENU = """
1) Registar chamada
2) Ler ficheiro
3) Listar clientes
4) Fatura
5) Terminar
Opção? """
```

```
# Função "principal":
# Tem o ciclo principal, que repete o menu e faz as operações escolhidas
def main():
    chamadas = {} # key=cliente (origem) value=lista de chamadas
    repetir = True
    while repetir:
        op = input(MENU)    # Lemos opção como string...

        if op == "1":       # ... e comparamos com outras strings
            chamada = inputCall()
            addCall(chamada, chamadas)
            #print(chamadas)     # print útil durante o desenvolvimento
        elif op == "2":
            fname = input("Ficheiro? ")
            loadCalls(fname, chamadas)
            #print(chamadas)     # print útil durante o desenvolvimento
        elif op == "3":
            clientes = sorted(chamadas.keys())
            # TPC: substitua o print por uma função que imprima os
clientes de
            # forma mais elegante (sem [], nem aspas, como no enunciado).
            print(clientes)
        elif op == "4":
            cliente = input("cliente? ")
            if cliente in chamadas:
                fatura(cliente, chamadas)
            else:
                print("Cliente desconhecido")
        elif op == "5":
            # Esta é uma forma de implementar a terminação: mudar uma
"flag"
            repetir = False
        else:
            # Outras opções são erro do utilizador!
            print("Opção inválida!")

    return

# Chamar função principal:
main()
```